Tugas Kelompok: Implementasi RAW, WAR, dan WAW pada Simulator ILP

INF-1316 Organisasi dan Arsitektur Komputer

Anggota	NIM
Siti Nor Hamidah (RAW)	51240022
Naila Nur Azizah Musgamy (WAR)	51240034
Siti Nahdia Nahda (WAW)	51240025

Link GitHub: https://github.com/hamidahnursiti09-ux/ilp-hazard-simulator

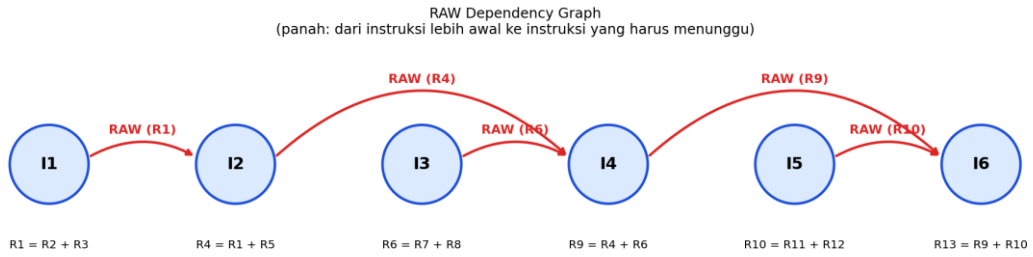
1. RAW (Read After Write)

RAW terjadi saat sebuah instruksi **membaca** register yang **ditulis** instruksi sebelumnya, jadi harus menunggu hasilnya. Ini true dependence, sehingga **tidak bisa** dihilangkan dengan register renaming.

Instruksi	Operasi	Instruksi	Operasi
I1	R1 = R2 + R3	I2	R4 = R1 + R5
I3	R6 = R7 + R8	I4	R9 = R4 + R6
I5	R10 = R11 + R12	I6	R13 = R9 + R10

RAW: I2 menunggu I1 (R1); I4 menunggu I2 (R4) dan I3 (R6); I6 menunggu I4 (R9) dan I5 (R10). Tidak ada WAR maupun WAW pada program ini.

Dependency graph



Hasil eksekusi

Skenario	Cycle	Speedup	Keterangan
Sequential	6	1.00x	Satu instruksi per cycle
ILP, RAW diabaikan	1 (tidak valid)	-	Salah: instruksi membaca nilai sebelum ditulis
ILP + penanganan RAW	4	1.50x	Benar: instruksi yang membaca menunggu penulisnya
ILP + register renaming	4	1.50x	Tetap sama, RAW tidak bisa dihilangkan

Jadwal ILP: Cycle 1: I1, I3, I5 | Cycle 2: I2 | Cycle 3: I4 | Cycle 4: I6

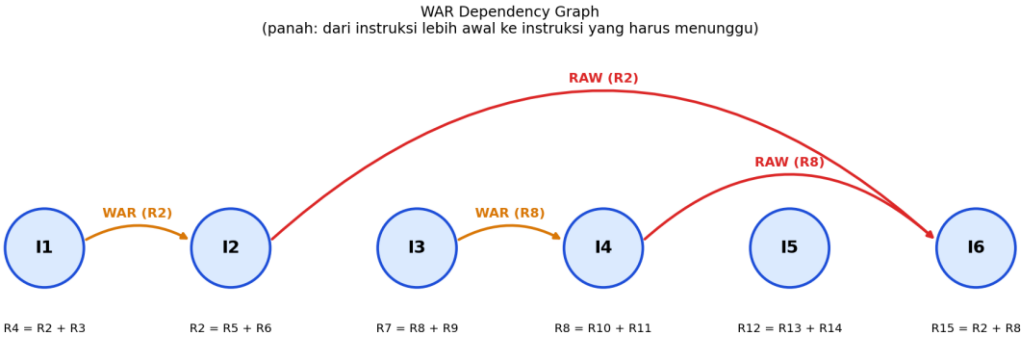
2. WAR (Write After Read)

WAR terjadi saat sebuah instruksi **menulis** register yang masih **dibaca** instruksi sebelumnya. Ini anti-dependence (name dependence), sehingga **bisa** dihilangkan dengan register renaming.

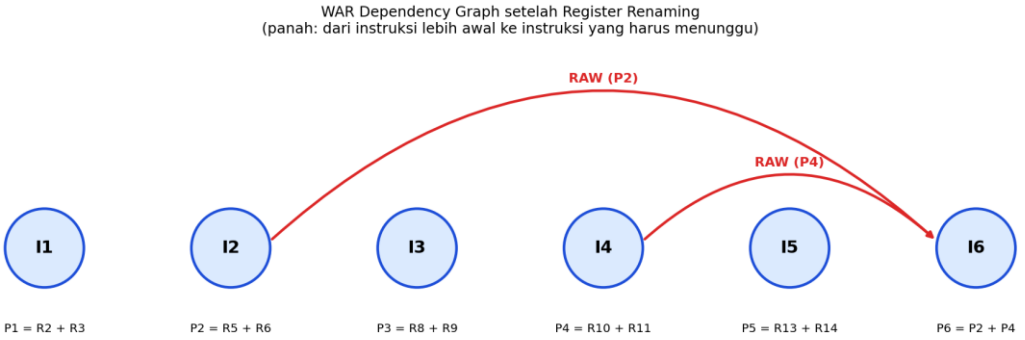
Instruksi	Operasi	Instruksi	Operasi
I1	R4 = R2 + R3	I2	R2 = R5 + R6
I3	R7 = R8 + R9	I4	R8 = R10 + R11
I5	R12 = R13 + R14	I6	R15 = R2 + R8

WAR: I2 menulis R2 yang dibaca I1; I4 menulis R8 yang dibaca I3. I6 membaca R2 dan R8 (RAW).

Dependency graph



Sesudah register renaming (hanya RAW yang tersisa):



Hasil eksekusi

Skenario	Cycle	Speedup	Keterangan
Sequential	6	1.00x	Satu instruksi per cycle
ILP, hanya cek RAW	2 (tidak valid)	-	Salah: penulis dan pembaca register sama jalan bersamaan
ILP + penanganan WAR	3	2.00x	Benar: I2 menunggu I1, I4 menunggu I3
ILP + register renaming	2	3.00x	Benar dan tercepat, WAR hilang

Jadwal ILP: Cycle 1: I1, I3, I5 | Cycle 2: I2, I4 | Cycle 3: I6

3. WAW (Write After Write)

WAW terjadi saat dua instruksi **menulis** ke register yang **sama**, jadi urutan penulisannya harus dijaga. Ini output dependence (name dependence), sehingga **bisa** dihilangkan dengan register renaming.

Instruksi	Operasi	Instruksi	Operasi
I1	R1 = R2 + R3	I2	R4 = R5 + R6
I3	R1 = R7 + R8	I4	R9 = R10 + R11
I5	R4 = R12 + R13	I6	R14 = R1 + R4

WAW: I1 dan I3 menulis R1; I2 dan I5 menulis R4. I6 membaca R1 dan R4 (RAW). Tidak ada WAR pada program ini.

Dependency graph



Kiri: sebelum register renaming. Kanan: sesudah renaming (hanya RAW yang tersisa).

Hasil eksekusi

Skenario	Cycle	Speedup	Keterangan
Sequential	6	1.00x	Satu instruksi per cycle
ILP, hanya cek RAW	2 (tidak valid)	-	Salah: dua penulis register sama jalan bersamaan
ILP + penanganan WAW	3	2.00x	Benar: I3 menunggu I1, I5 menunggu I2
ILP + register renaming	2	3.00x	Benar dan tercepat, WAW hilang

Jadwal ILP: Cycle 1: I1, I2, I4 | Cycle 2: I3, I5 | Cycle 3: I6

4. Kesimpulan

Hazard	Sequential	ILP + penanganan	ILP + renaming	Hilang dengan renaming?
RAW	6 cycle	4 cycle	4 cycle	Tidak
WAR	6 cycle	3 cycle	2 cycle	Ya
WAW	6 cycle	3 cycle	2 cycle	Ya

RAW tidak bisa dihilangkan karena ada aliran data yang nyata, jadi instruksi pembaca tetap menunggu. WAR dan WAW hanya bentrok nama register, sehingga hilang setelah register renaming (3 cycle menjadi 2 cycle pada program uji). Kalau simulator hanya mengecek RAW, program WAR dan WAW kelihatan lebih cepat (2 cycle) tetapi hasilnya salah.